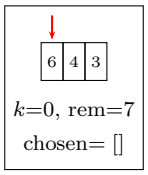
Lecture Slip, CS 235
Oct 29, 2026, Thursday
Pruning the Search Tree

Name:	
EmpID:	

1. **Pruning and early termination.** Here is `find` from class. It asks whether some subset of `v[k..]` adds up to `rem`.

```
bool find(const std::vector<int>& v, int k, int rem, std::vector<int>& chosen) {
  if (rem == 0) { return true; }                                  // goal
  if (k == static_cast<int>(v.size()) || rem < 0) { return false; } // dead end
  chosen.push_back(v[k]);                                         // choose
  if (find(v, k + 1, rem - v[k], chosen)) { return true; }        // take
  chosen.pop_back();                                              // undo
  return find(v, k + 1, rem, chosen);                             // skip
}
// v = {6, 4, 3}; chosen starts empty; first call: find(v, 0, 7, chosen)
```

For the recursive call tree in part (c), draw each call as a **node** like the example below: show the array `v` with an arrow at `v[k]`, followed by `k`, `rem`, and `chosen`.



(a) List every call in order as a triple (`k`, `rem`, `chosen`). How many calls are there?

(b) Which calls are stopped because `rem < 0`? What is `chosen` when the first call returns?

(c) Now let `v = {4, 3, 2, 1}` and make the first call `find(v, 0, 5, chosen)`. Draw the recursive call tree with nodes as above: label each edge take or skip, and number the calls in order. How many calls are there, and what is `chosen` when the first call returns?

2. **A bound.** Let `v = {5, 4, 3, 2, 1}` and `n = v.size()`. Define `suffix[k]` as the sum of `v[k..4]`, with `suffix[5] = 0`. The pruning test `rem > suffix[k]` is added, so the dead-end line becomes `if (k == n || rem < 0 || rem > suffix[k]) return false;`

   (a) Write the array `suffix[0..5]`.

   (b) With target 20, how many calls does the search make? With target 9, how many calls, and what is `chosen`?

3. **Counting Make 24 moves.** In class, a move picks an **unordered pair** and replaces it by one result, with 6 possible results per pair ($a+b,\ a\times b,\ a-b,\ b-a,\ a\div b,\ b\div a$). For counting purposes, assume all six operations are available at every step.

   (a) With **5 numbers** on the table, how many first moves are there?

   (b) Starting from 5 numbers, how many complete sequences of moves are possible under this assumption? Write the product.

4. **Duplicate letters.** `printUnique` from class skips a letter that was already tried at the same position. Let `std::string s = "LEVEL";` be the input.

   (a) How many lines would the original `printAll(s, 0)` print? How many does `printUnique(s, 0)` print?

   (b) Why does the original print so many more lines?

5. Do you have any questions about today's class? If so, please provide details. Thank you.